

TP : Debug d'un Processus Actif

Durée estimée : 2-3 heures

Objectif : Attacher un debugger à un processus en cours, placer des breakpoints, inspecter et modifier le comportement à la volée

Prérequis

Outils nécessaires (Linux)

```
# Installation des outils de base
sudo apt-get update
sudo apt-get install gdb build-essential

# Vérifier l'installation
gdb --version
```

Étape 1 : Compilation et Lancement du Programme Cible

Compilation du programme

Un programme C simulant une application avec login est fourni : `active_target.c`

```
# Compiler le programme
make

# Ou manuellement
gcc -g -fno-pie -no-pie -o active_target active_target.c
```

Note sur les flags de compilation :

- `-g` : Inclut les symboles de debug (essentiel pour GDB)
- `-fno-pie -no-pie` : Désactive PIE pour des adresses fixes

Lancement du programme

```
# Lancer l'application
./active_target
```

Le programme affiche son **PID** et simule un serveur d'authentification qui vérifie des credentials toutes les 5 secondes.

EXERCICE 1.1

1. Compilez le programme avec `make`
2. Lancez-le avec `./active_target`
3. Observez les tentatives de login échouées
4. Notez le PID affiché

 Notez le PID ici : _____

Le programme :

- Vérifie un login/password toutes les 5 secondes
- Le mot de passe correct est `SecretPass123!`
- Affiche "❌ Login failed!" tant que le mauvais password est utilisé
- Affiche "✅ Login successful!" avec le bon password

Étape 2 : Attacher GDB au Processus

Méthode 1 : Attacher au processus en cours

```
# Dans un AUTRE terminal
sudo gdb -p PID

# Exemple si PID = 12345
sudo gdb -p 12345
```

Méthode 2 : Lancer directement avec GDB

```
# Alternative : lancer le programme depuis GDB
gdb ./active_target

# Dans GDB
(gdb) run
```

EXERCICE 2.1

1. Le programme tourne avec des login qui échouent
2. Dans un nouveau terminal, attachez GDB au processus
3. Vérifiez que GDB est bien attaché

Commandes :

```
# Terminal 1 : programme en cours
./active_target

# Terminal 2 : attacher GDB
```

```
sudo gdb -p VOTRE_PID
(gdb) fs cmd
```

Dans GDB, vous verrez :

```
Attaching to process 12345
Reading symbols from /path/to/active_target...
```



GDB s'est-il attaché correctement ? _____

Étape 3 : Placement de Breakpoints

3.1 Lister les fonctions disponibles

```
# Dans GDB
(gdb) info functions

# Chercher des fonctions spécifiques
(gdb) info functions check
```

3.2 Placer des breakpoints

```
# Breakpoint sur la fonction check_password
(gdb) break check_password
(gdb) info breakpoints

# Breakpoint sur une ligne spécifique
(gdb) break active_target.c:25

# Breakpoint conditionnel
(gdb) break check_password if attempt > 3
```



EXERCICE 3.1

Placez un breakpoint sur la fonction `check_password` :

```
(gdb) break check_password
(gdb) continue
```

Attendez quelques secondes... Le programme va s'arrêter au breakpoint !

Dans GDB, vous verrez :

```
Breakpoint 1, check_password () at active_target.c:XX
XX      int result = strcmp(password, correct_password);
```

 **Le breakpoint a-t-il été déclenché ?** _____

3.3 Gérer les breakpoints

```
# Lister tous les breakpoints
(gdb) info breakpoints

# Désactiver un breakpoint
(gdb) disable 1

# Réactiver un breakpoint
(gdb) enable 1

# Supprimer un breakpoint
(gdb) delete 1

# Supprimer tous les breakpoints
(gdb) delete
```



Étape 4 : Inspection de la Mémoire et des Variables

4.1 Examiner les variables locales

```
# Afficher toutes les variables locales
(gdb) info locals

# Afficher une variable spécifique
(gdb) print password
(gdb) print correct_password
(gdb) print attempt

# Format différent
(gdb) print/x attempt    # hexadécimal
(gdb) print/d attempt    # décimal
(gdb) print/s password   # string
```

4.2 Examiner les registres

```
# Tous les registres
(gdb) info registers
```

```
# Registre spécifique
(gdb) info register rip
(gdb) info register rsp
```

4.3 Examiner la mémoire

```
# Examiner une adresse mémoire
(gdb) x/s &password          # Afficher comme string
(gdb) x/20c &password        # Afficher 20 caractères
(gdb) x/4x $rsp               # 4 mots en hexadécimal depuis le stack
pointer

# Format : x/[nombre][format][taille] adresse
# Format : s (string), x (hex), d (decimal), i (instruction)
# Taille : b (byte), h (half-word), w (word), g (giant, 8 bytes)
```

EXERCICE 4.1

Au breakpoint, inspectez les variables :

```
# Vous êtes arrêté à check_password()
(gdb) info locals
(gdb) print password
(gdb) print correct_password
(gdb) print attempt
```

Questions :

- Quelle est la valeur de `password` ? _____
- Quelle est la valeur de `correct_password` ? _____
- Quelle est la valeur de `attempt` ? _____

4.4 Examiner la stack

```
# Voir la stack trace (call stack)
(gdb) backtrace
(gdb) bt

# Voir les frames
(gdb) info frame
(gdb) frame 0

# Remonter dans la stack
(gdb) up
(gdb) down
```



Étape 5 : Modification du Comportement à la Volée

5.1 Modifier des variables

```
# Changer la valeur d'une variable
(gdb) set variable attempt = 999
(gdb) print attempt

# Modifier un string
(gdb) set password = "SecretPass123!"
(gdb) print password
```

5.2 Modifier des registres

```
# Changer un registre
(gdb) set $rax = 0

# Forcer un retour de fonction
(gdb) set $rax = 1
(gdb) return 1
```

5.3 Patcher la mémoire

```
# Modifier directement une adresse mémoire
(gdb) set {int}0xADDRESSE = 42
(gdb) set {char}0xADDRESSE = 'A'

# Modifier plusieurs bytes
(gdb) set *(int*)0xADDRESSE = 0x90909090
```



EXERCICE 5.1 : Bypasser la vérification de mot de passe

Méthode 1 : Modifier la variable **result**

Au breakpoint dans **check_password()**, après la ligne **strcmp** :

```
# Forcer result à 0 (strings égaux)
(gdb) set variable result = 0

(gdb) print result

# Continuer l'exécution
(gdb) continue
```

```
# Ou alors
(gdb) n
```

 Que se passe-t-il dans le terminal du programme ? _____

Méthode 2 : Modifier directement le password

```
# Au breakpoint, avant strcmp
(gdb) set password = "SecretPass123!"
(gdb) print password
(gdb) continue
```

Méthode 3 : Modifier le compteur d'essais

```
# Forcer le compteur à une valeur élevée
(gdb) set variable attempt = 100
(gdb) continue
```

5.4 Appeler des fonctions

```
# Appeler une fonction directement
(gdb) call printf("Hello from GDB!\n")
(gdb) call check_password()

# Appeler avec des arguments
(gdb) call strcmp("test", "test")
```

Étape 6 : Contrôle de l'Exécution

6.1 Commandes de base

```
# Continuer l'exécution
(gdb) continue
(gdb) c

# Exécuter une instruction
(gdb) step          # Entre dans les fonctions
(gdb) s

(gdb) next          # Saute les appels de fonction
(gdb) n

# Exécuter jusqu'à la fin de la fonction
```

```
(gdb) finish

# Exécuter jusqu'à une ligne
(gdb) until 50
```

6.2 Watchpoints (surveillance de variables)

```
# Surveiller une variable
(gdb) watch attempt
(gdb) watch password

# S'arrêter quand la variable change
# Le programme s'arrêtera automatiquement quand 'attempt' est modifié

# Lister les watchpoints
(gdb) info watchpoints
```

EXERCICE 6.1

Placez un watchpoint sur `attempt` et observez :

```
# Reprendre depuis le début
(gdb) kill
(gdb) run

# Placer un watchpoint
(gdb) watch attempt

# Continuer
(gdb) continue
```

Le programme s'arrêtera chaque fois que `attempt` change !

 **Combien de fois le watchpoint s'est-il déclenché ? _____**

Étape 7 : Désassemblage et Analyse (Avancées)

7.1 Désassembler du code

```
# Désassembler la fonction actuelle
(gdb) disassemble

# Désassembler une fonction spécifique
(gdb) disassemble check_password
(gdb) disas main
```



```
# Voir le code assembleur avec le code source
(gdb) set disassemble-next-line on
(gdb) show disassemble-next-line
```

7.2 Examiner les instructions

```
# Afficher les prochaines instructions
(gdb) x/10i $rip

# Layout avec assembleur et registres (TUI mode)
(gdb) layout asm
(gdb) layout regs
(gdb) layout split

# Quitter le layout
(gdb) tui disable
# ou Ctrl+X puis A
```

EXERCICE 7.1

Désassemblez la fonction `check_password` :

```
(gdb) break check_password
(gdb) run
(gdb) disassemble
```

Questions :

- Quelle instruction appelle `strcmp` ? _____
- Quelle instruction teste le résultat ? _____ (cherchez `cmp` ou `test`)

7.3 Modifier le flux d'exécution

```
# Sauter à une adresse spécifique
(gdb) jump *0xADDRESSE

# Sauter à une ligne
(gdb) jump 42

# Définir le prochain instruction pointer
(gdb) set $rip = 0xADDRESSE
```

Challenges Avancés

Challenge 1 : Bypass permanent

Objectif : Modifier le binaire pour qu'il accepte n'importe quel password

Indices :

1. Trouvez l'adresse de l'instruction `cmp` ou `test` après `strcmp`
2. Remplacez-la par des `nop` (0x90)
3. Utilisez `set {unsigned char}0xADDRESSE = 0x90`

Commandes :

```
(gdb) disassemble check_password
# Trouvez l'adresse de la comparaison
(gdb) x/5i 0xADDRESSE_DE_LA_COMPARAIISON
# Remplacez par des NOPs
(gdb) set {unsigned char}0xADDRESSE = 0x90
(gdb) set {unsigned char}0xADDRESSE+1 = 0x90
```

Challenge 2 : Injection de code

Objectif : Appeler une fonction "cachée" qui n'est jamais appelée normalement

Le programme contient une fonction `secret_admin_mode()` qui n'est jamais appelée.

```
# Trouver l'adresse de la fonction
(gdb) info functions secret
(gdb) print secret_admin_mode

# L'appeler directement
(gdb) call secret_admin_mode()
```

Challenge 3 : Race condition

Objectif : Synchroniser avec le programme pour modifier `attempt` au bon moment

```
# Placer un watchpoint sur attempt
(gdb) watch attempt

# Modifier attempt juste après son incrémentation
(gdb) set variable attempt = 1000

# Observer le message spécial qui s'affiche
```

Challenge 4 : Memory mapping

Objectif : Explorer les régions mémoire du processus

```
# Voir toutes les zones mémoire
(gdb) info proc mappings

# Examiner le heap
(gdb) info proc mappings
# Trouvez la ligne [heap]
(gdb) x/100s 0xADDRESSE_HEAP

# Examiner le stack
(gdb) x/100x $rsp
```



Commandes Utiles GDB (Aide-Mémoire)

Démarrage et attachement

```
gdb ./programme          # Lancer avec GDB
gdb -p PID                # Attacher à un processus
gdb -q                   # Mode quiet (sans banner)
```

Breakpoints

```
break fonction           # BP sur fonction
break fichier.c:42       # BP sur ligne
break *0x400500           # BP sur adresse
info breakpoints         # Lister les BP
delete 1                 # Supprimer BP #1
```

Exécution

```
run                      # Lancer le programme
continue                 # Continuer
step                     # Pas à pas (entre dans les fonctions)
next                     # Pas à pas (saute les fonctions)
finish                   # Terminer la fonction courante
```

Inspection

```
info locals              # Variables locales
print variable           # Afficher une variable
print/x variable        # En hexadécimal
x/s 0xADDRESSE          # Examiner mémoire (string)
```

```
info registers      # Tous les registres
backtrace          # Call stack
```

Modification

```
set variable var = 42      # Changer une variable
set $rax = 0              # Changer un registre
call fonction()            # Appeler une fonction
```

Désassemblage

```
disassemble fonction      # Désassembler
layout asm                # Vue assembleur (TUI)
layout split               # Vue mixte source + asm
```

Autres

```
help commande            # Aide sur une commande
quit                     # Quitter GDB
detach                   # Se détacher du processus (le laisse tourner)
```



Exercices Récapitulatifs

Exercice Final : Scénario complet

Situation : Vous avez un programme en production qui refuse votre login. Vous devez déboguer le processus actif pour comprendre pourquoi et le corriger à la volée sans le redémarrer.

Étapes :

1. ☒ Lancer le programme et noter le PID
2. ☒ Attacher GDB au processus
3. ☒ Placer un breakpoint sur `check_password`
4. ☒ Attendre que le breakpoint se déclenche
5. ☒ Inspecter les variables `password` et `correct_password`
6. ☒ Modifier `result` pour bypasser la vérification
7. ☒ Vérifier que "Login successful!" s'affiche
8. ☒ Détacher GDB proprement
9. ☒ Vérifier que le programme continue de tourner normalement

Commandes complètes :

```
# Terminal 1
./active_target

# Terminal 2
sudo gdb -p PID_DU_PROGRAMME
```

```
(gdb) break check_password
(gdb) continue
# ... attendre le breakpoint ...
(gdb) info locals
(gdb) print password
(gdb) print correct_password
(gdb) next
(gdb) print result
(gdb) set variable result = 0
(gdb) continue
# Observer "Login successful!" dans le terminal 1
(gdb) detach
(gdb) quit
```

Conclusion

Vous avez appris à :

-  Attacher un debugger à un processus actif
-  Placer et gérer des breakpoints
-  Inspecter la mémoire, les registres et les variables
-  Modifier le comportement d'un programme à la volée
-  Contrôler le flux d'exécution
-  Désassembler et analyser du code en temps réel

Ces compétences sont essentielles pour :

-  Débugger des applications en production
-  Analyser des malwares
-  Faire du reverse engineering
-  Tester la sécurité d'applications
-  Corriger des bugs sans redémarrage

Ressources

- [GDB Documentation](#)
- [GDB Cheat Sheet](#)
- [Debugging with GDB \(GNU Manual\)](#)

Bon debugging ! 🚀